

Deloitte – Data Engineer Interview Q&A

Q1. Explain the architecture of Azure Data Factory and its integration runtime.

A1. Azure Data Factory (ADF) follows a pipeline-based architecture where you design ETL workflows. It uses **pipelines** (workflow of activities), **datasets** (represent data structures), and **linked services** (connection information). The **Integration Runtime (IR)** is the compute infrastructure used for data movement and activity dispatch. ADF supports three IR types: **Azure IR** (cloud-based), **Self-hosted IR** (on-premises or private network), and **SSIS IR** (to run existing SSIS packages). This architecture allows hybrid data integration across on-prem and cloud systems.

Q2. How do you design a star schema for reporting in a data warehouse?

A2. A star schema consists of **fact tables** (containing measures like sales, revenue, quantity) connected to **dimension tables** (such as customer, product, date, location). To design:

1. Identify business processes and key measures.
2. Create fact tables for metrics.
3. Design dimension tables with descriptive attributes.
4. Define relationships with surrogate keys.

This design improves query performance because BI tools can directly join facts and dimensions with fewer joins than snowflake schema.

Q3. What is the difference between Delta Lake and Parquet in Databricks?

A3. **Parquet** is a file format that stores data in a columnar manner but does not provide transaction support. **Delta Lake** is built on top of Parquet and adds features like **ACID transactions, schema enforcement, time travel, and upserts/merges**. In practice, Delta Lake ensures consistency and reliability in big data pipelines, whereas Parquet alone is a storage format without governance or transactional guarantees.

Q4. How do you handle schema drift in Azure Synapse pipelines?

A4. Schema drift refers to changes in source schema (columns added/removed). In Synapse pipelines, you can handle it by:

- Enabling **Schema Drift** in Data Flows, which allows flexible mapping.
- Using **wildcard mappings** and dynamic column handling.

- Implementing **data lake storage with schema evolution** (Delta Lake). Additionally, maintaining metadata-driven frameworks helps adjust mappings automatically as schema evolves.
-

Q5. Explain PolyBase and its use in Synapse Analytics.

A5. PolyBase is a technology in Synapse that enables querying external data (e.g., Blob, ADLS, Hadoop) using **T-SQL** without moving it into the database. It allows parallel data reads into Synapse using distributed computing. Common use cases:

- Querying raw files in data lake directly.
 - Loading large amounts of external data into Synapse.
 - Integrating on-premises and cloud data sources efficiently.
-

Q6. How do you optimize performance for ETL pipelines in Azure Data Factory?

A6. Performance can be optimized by:

- Using **parallelism** and partitioning for large loads.
 - Choosing the correct **integration runtime size**.
 - Leveraging **staging with PolyBase** for Synapse bulk loads.
 - Applying **filtering and incremental loads** instead of full reloads.
 - Monitoring with **Azure Monitor** and adjusting pipeline concurrency.
- Following these ensures scalable, cost-efficient ETL processes.
-

Q7. What is partitioning in Spark? Explain with an example.

A7. Partitioning divides data into smaller chunks distributed across executors for parallelism. For example, a 1TB dataset can be partitioned into 200 parts to allow distributed computation. You can control partitioning using `repartition()` or `coalesce()`. For instance:

```
df = df.repartition(50, "country")
```

This ensures data is distributed by country, improving join and aggregation performance.

Q8. How do you implement incremental data loads in a data pipeline?

A8. Incremental loads capture only new or changed data rather than reloading everything. Techniques include:

- Using **watermark columns** like LastModifiedDate.
 - Implementing **Change Data Capture (CDC)** or **Delta Lake MERGE** operations.
 - Storing high-watermark in metadata tables and filtering sources accordingly.
This reduces load time, optimizes resources, and keeps data pipelines efficient.
-

Q9. Explain Slowly Changing Dimensions (SCD) Types 1, 2, and 3.

A9.

- **Type 1:** Overwrites old data (no history). E.g., correcting spelling of a customer name.
 - **Type 2:** Maintains full history by creating a new record with start/end dates or flags. E.g., tracking customer address changes.
 - **Type 3:** Keeps limited history by adding a new column for previous values.
SCD Type 2 is most common for historical analytics.
-

Q10. How do you handle late-arriving data in a data pipeline?

A10. Techniques include:

- **Watermarking** to process late events up to a threshold.
 - Using **Delta Lake MERGE** for upserts.
 - Staging late data separately and merging with fact tables.
 - Implementing **event-time windows** in streaming.
This ensures data accuracy without duplications.
-

Q11. Difference between Azure Data Lake Storage Gen1 and Gen2.

A11.

- **Gen1:** Hierarchical namespace, limited integration, less secure.
 - **Gen2:** Combines **Blob storage scalability** + **Gen1 hierarchical namespace**.
Supports **RBAC, ACLs, cheaper storage, and higher performance**.
Today, Gen2 is the standard choice.
-

Q12. Explain Azure Key Vault integration in Data Factory pipelines.

A12. Azure Key Vault securely stores credentials, secrets, and keys. ADF pipelines can link Key Vault via **linked service**, and fetch secrets dynamically using **reference syntax**. This avoids hardcoding credentials in pipelines, ensuring security and compliance.

Q13. How do you ensure data quality and validation in ETL workflows?

A13. Strategies include:

- Implementing **data profiling** and validation rules (null checks, data type checks).
 - Using **assertions** in ADF data flows or PySpark.
 - Creating **error-handling sinks** for bad data.
 - Automating validation against metadata and business rules.
This guarantees reliable data for analytics.
-

Q14. What are the best practices for securing data pipelines in Azure?

A14.

- Use **Managed Identities** for authentication.
 - Store secrets in **Key Vault**.
 - Restrict access with **RBAC** and network rules.
 - Encrypt data in-transit (HTTPS) and at-rest (Azure encryption).
 - Enable **audit logs** and monitor with Sentinel.
These ensure compliance and security.
-

Q15. Explain triggers in ADF and their types.

A15. Triggers start pipeline executions. Types include:

- **Schedule Trigger** – runs on defined time intervals.
 - **Tumbling Window Trigger** – runs in fixed-size intervals, supports retry and backfill.
 - **Event-based Trigger** – responds to events like Blob creation.
They enable automation of data workflows.
-

Q16. What is the difference between Data Flow and Pipeline in ADF?

A16.

- **Pipeline:** High-level orchestration of activities (data copy, transformations, external tasks).
 - **Data Flow:** Visual transformation logic executed on a Spark cluster (joins, aggregations, filters).
- Pipelines can include Data Flows as activities, but not vice versa.
-

Q17. Explain Unity Catalog in Databricks.

A17. Unity Catalog is a **centralized data governance solution** in Databricks. It provides fine-grained access control at **catalog, schema, table, column** level, with audit logs. It also supports data lineage, making it easier to track data usage across Databricks workspaces.

Q18. How would you design a data lakehouse for enterprise analytics?

A18. A lakehouse combines **data lake flexibility** with **data warehouse performance**.

Design steps:

- Raw zone for unprocessed data.
 - Cleansed/bronze-silver-gold (Medallion) architecture using Delta Lake.
 - Implement governance with Unity Catalog.
 - Expose curated data via Synapse/Power BI.
- This ensures scalability, cost-effectiveness, and governance.
-

Q19. How do you handle job failures and retries in ADF?

A19. ADF supports retry policies at activity level. Best practices:

- Enable **retry count & interval** in activities.
 - Use **tumbling window trigger with retry** for scheduled jobs.
 - Log failures with **Log Analytics**.
 - Send alerts via Logic Apps or email.
- This ensures resilience in production pipelines.
-

Q20. Explain the difference between linked service and dataset in ADF.

A20.

- **Linked Service:** Connection information (e.g., credentials, server details).
 - **Dataset:** Schema/structure of data within a linked service.
Example: Linked Service = Azure SQL DB connection; Dataset = Sales table inside that DB.
-

Q21. What are different cluster types in Azure Databricks?

A21.

- **Interactive Clusters:** For development & debugging.
 - **Job Clusters:** Spin up for a job, then terminate.
 - **High Concurrency Clusters:** Optimized for multiple users and BI tools.
Choosing depends on workload type and cost efficiency.
-

Q22. How do you debug and monitor Spark jobs in Databricks?

A22.

- Use **Spark UI** to analyze DAGs, stages, tasks.
 - Check **logs in Databricks cluster UI**.
 - Enable **Ganglia metrics**.
 - Set up **Azure Monitor** for alerts.
 - Use **structured logging** for traceability.
This helps identify skew, shuffle issues, and long-running tasks.
-

Q23. Explain Delta Live Tables (DLT).

A23. Delta Live Tables is a Databricks framework for **declarative ETL pipelines**. You define transformations as queries, and DLT manages orchestration, dependencies, retries, and quality checks. It ensures reliability and simplifies managing streaming + batch pipelines.

Q24. How do you handle GDPR and compliance in data engineering?

A24.

- Anonymize or pseudonymize PII.

- Implement **data masking and tokenization**.
 - Track **data lineage** for auditing.
 - Use **RBAC and Key Vault** for access control.
 - Respect **data retention policies**.
This ensures compliance with GDPR and other regulations.
-

Q25. What are the differences between Azure Synapse Dedicated and Serverless pools?

A25.

- **Dedicated Pool:** Pre-provisioned compute (DWU-based), best for predictable workloads.
 - **Serverless Pool:** Pay-per-query model using distributed on-demand compute, best for ad-hoc exploration.
Dedicated = high performance, predictable cost; Serverless = flexibility, low upfront cost.
-

Q26. Explain the lifecycle of a Spark application.

A26.

1. **Submit:** User submits job using spark-submit.
 2. **Driver:** Coordinates execution and creates SparkContext.
 3. **Cluster Manager:** Allocates resources (YARN, Kubernetes, Databricks).
 4. **Executors:** Run tasks on worker nodes.
 5. **Stages/Tasks:** DAG splits into tasks executed in parallel.
 6. **Completion:** Results returned to driver or written to sink.
-

Q27. How do you tune performance in Spark transformations?

A27.

- Use **partitioning** and **bucketing** for joins.
- Cache intermediate results when reused.
- Avoid shuffles by using broadcast joins.
- Repartition data optimally.

- Monitor with Spark UI and optimize skew.
This reduces execution time significantly.
-

Q28. Explain CI/CD setup for ADF pipelines in Azure DevOps.

A28.

1. Store ADF pipelines in Git repo.
 2. Create **build pipeline** for validation.
 3. Create **release pipeline** for deployment to higher environments.
 4. Use ARM templates or ADF publish branch for deployments.
This ensures controlled, automated deployments.
-

Q29. What are the common challenges faced in ETL design?

A29.

- Handling schema drift and evolving sources.
 - Managing large volumes (performance optimization).
 - Ensuring data quality & governance.
 - Building fault tolerance.
 - Cost management in cloud.
- Overcoming these requires metadata-driven, modular ETL frameworks.
-

Q30. Explain Medallion Architecture in Azure Databricks.

A30. Medallion Architecture is a layered design for lakehouses:

- **Bronze Layer:** Raw ingestion (as-is).
 - **Silver Layer:** Cleaned, enriched, deduplicated data.
 - **Gold Layer:** Aggregated, business-ready data.
- This approach ensures reliability, scalability, and governance for analytics.
-

PwC – Data Engineer Interview Q&A

Q1. Explain your experience with Azure Data Factory and ETL development.

A1. Azure Data Factory (ADF) enables building scalable ETL pipelines. You can create

pipelines to orchestrate data movement, use **Copy Activity** for ingestion, and apply **Mapping Data Flows** for transformations. ADF supports both batch and near real-time ETL. In real projects, ETL pipelines often involve extracting from SQL/Blob sources, transforming using ADF Data Flows or Databricks, and loading into Synapse or Power BI models.

Q2. How do you design a data model for reporting?

A2. Reporting models are designed using **star schema** or **snowflake schema**. The fact table contains measures (sales, revenue), while dimensions store descriptive data (date, customer, product). Keys link facts to dimensions. Good models also include surrogate keys, hierarchies, and aggregated fact tables. This design improves query performance and supports BI tools like Power BI.

Q3. Difference between OLTP and OLAP.

A3.

- **OLTP (Online Transaction Processing):** Optimized for high-volume inserts/updates, normalized schema, real-time business transactions (e.g., banking system).
 - **OLAP (Online Analytical Processing):** Optimized for analytical queries, denormalized star schema, used for reporting and data warehousing. ETL pipelines often move data from OLTP → OLAP systems for analytics.
-

Q4. How do you handle large datasets in PySpark?

A4.

- Use **partitioning** and **bucketing** for parallelism.
 - Apply **lazy evaluation** and optimize actions.
 - Cache data when reused across queries.
 - Use **filter pushdown** and avoid collecting to driver.
 - Handle skew using salting or repartitioning.
This ensures PySpark jobs scale to TB/PB-level datasets efficiently.
-

Q5. Explain caching and persistence in Spark.

A5. Caching stores intermediate DataFrames in memory for reuse. Persistence allows specifying storage levels (MEMORY_ONLY, DISK_ONLY, MEMORY_AND_DISK). Example:

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

This improves performance when a dataset is reused multiple times across transformations.

Q6. How do you set up CI/CD for data pipelines?

A6.

1. Store pipeline code (ADF JSON, Databricks notebooks, PySpark) in Git.
 2. Configure **build pipeline** in Azure DevOps to validate.
 3. Use **release pipeline** to deploy via ARM templates or APIs.
 4. Apply environment-specific configurations using parameters.
This ensures automated, version-controlled deployments.
-

Q7. Explain incremental load strategies in ADF.

A7. Incremental loads reduce processing by capturing only changes. Strategies:

- Use **watermark columns** (e.g., last modified timestamp).
 - Implement **CDC** from source systems.
 - Leverage **Delta Lake MERGE** for upserts.
 - Maintain metadata to track high-watermarks.
This ensures faster, cost-efficient pipelines.
-

Q8. How do you handle schema evolution in Delta Lake?

A8. Delta Lake supports schema evolution with:

- mergeSchema option when writing new data.
 - ALTER TABLE ADD COLUMNS for structural changes.
 - Managing backward compatibility by versioning.
This makes pipelines resilient to source schema changes.
-

Q9. What is data partitioning and bucketing in Spark?

A9.

- **Partitioning:** Divides data into logical chunks for parallelism (e.g., by region).

- **Bucketing:** Distributes data into fixed buckets using hash function (e.g., customer_id). Partitioning optimizes scanning, while bucketing improves join performance by colocating keys.
-

Q10. Explain triggers and parameterization in ADF.

A10.

- **Triggers:** Automate pipeline execution (schedule, tumbling window, event-based).
 - **Parameterization:** Pass dynamic values into pipelines, datasets, or linked services. Example: using parameterized file names for dynamic ingestion. This allows reusability and automation of pipelines.
-

Q11. What is the difference between Control Flow and Data Flow in ADF?

A11.

- **Control Flow:** Orchestration layer (activities, dependencies, error handling).
 - **Data Flow:** Transformation layer (joins, filters, aggregations) executed on Spark clusters. Together, they form complete ETL pipelines.
-

Q12. Explain the concept of PolyBase.

A12. PolyBase allows Synapse to query external data stored in Blob, ADLS, or Hadoop using T-SQL. It supports high-performance parallel reads, enabling bulk data movement and external table querying without copying data into Synapse first.

Q13. How do you design fault-tolerant data pipelines?

A13.

- Enable **retry policies** and error-handling sinks.
 - Use **idempotent loads** (safe re-runs without duplicates).
 - Implement **checkpointing** in streaming.
 - Design **metadata-driven pipelines** for flexibility. This ensures pipelines recover gracefully from failures.
-

Q14. What are different types of joins in Spark?

A14.

- **Inner Join:** Matches records from both datasets.
 - **Left/Right Outer Join:** Preserves all rows from one side.
 - **Full Outer Join:** Combines all rows from both datasets.
 - **Cross Join:** Cartesian product.
 - **Broadcast Join:** Small dataset broadcasted to all nodes.
Choice depends on dataset size and use case.
-

Q15. How do you handle duplicate records in Spark?

A15.

- Use `dropDuplicates()` with key columns.
 - Apply window functions with `ROW_NUMBER()` to keep latest.
 - Deduplicate during ingestion with Delta Lake `MERGE`.
This ensures data accuracy in fact tables.
-

Q16. Explain how you'd optimize a Spark SQL query.

A16.

- Enable **predicate pushdown**.
 - Partition data by frequently filtered columns.
 - Use **broadcast joins** for small dimension tables.
 - Avoid UDFs; use built-in functions.
 - Cache intermediate results when reused.
This reduces shuffle and improves query execution.
-

Q17. What are common performance bottlenecks in ADF pipelines?

A17.

- Using small batch sizes in Copy Activity.
- Lack of parallelism or partitioning.
- Complex transformations inside ADF instead of Spark.

- High latency in self-hosted IR.
Optimizations include tuning batch size, using staging, and leveraging parallel loads.
-

Q18. How do you schedule pipelines in Azure?

A18. Pipelines can be scheduled using **Schedule triggers** (cron-like), **Tumbling window triggers** (with backfill/retry), or **Event-based triggers** (on blob/file arrival). For enterprise scheduling, integration with Azure Logic Apps or Event Grid is also common.

Q19. Explain data lineage tracking in data pipelines.

A19. Data lineage shows data flow from source to destination. In ADF, lineage can be tracked using **integration with Purview**, activity logs, or custom metadata tables. Unity Catalog in Databricks also provides column-level lineage, ensuring auditability.

Q20. How do you secure sensitive credentials in Azure?

A20.

- Store credentials in **Azure Key Vault**.
 - Use **Managed Identity** for authentication.
 - Apply **RBAC** and firewall restrictions.
 - Mask secrets in logs.
This prevents hardcoding of credentials and ensures compliance.
-

Q21. Explain Managed vs External tables in Databricks.

A21.

- **Managed Table:** Databricks manages both metadata and data location. Dropping the table deletes data.
 - **External Table:** Metadata stored in metastore but data resides at external location. Dropping the table does not delete data.
External tables are preferred for data lake integration.
-

Q22. What is Z-ordering in Delta Lake?

A22. Z-ordering is a technique that colocates related records on disk using multi-dimensional clustering. For example, Z-ordering on customer_id and date improves

query performance when filtering by those columns. It reduces scan overhead and speeds up queries.

Q23. How do you enforce governance in Databricks Unity Catalog?

A23. Unity Catalog enforces governance by:

- Role-based access control at catalog/schema/table level.
 - Column-level security and masking.
 - Centralized auditing and lineage.
 - Integration with Azure AD for SSO.
- This ensures compliance and fine-grained access management.
-

Q24. Explain SCD Type 2 implementation in ADF.

A24. In ADF, SCD Type 2 can be implemented with:

1. Staging data from source.
 2. Joining with target table to detect changes.
 3. Expiring old records (set end_date).
 4. Inserting new record with updated attributes.
- Mapping Data Flows in ADF provide built-in SCD components.
-

Q25. How do you validate data in ETL workflows?

A25.

- Apply row counts and checksum validation.
 - Use business rules (e.g., revenue > 0).
 - Compare source vs target aggregates.
 - Create error handling flows for invalid rows.
- This ensures accuracy and trust in ETL outputs.
-

Q26. What is the difference between data lake and data warehouse?

A26.

- **Data Lake:** Stores raw structured + unstructured data at low cost, schema-on-read.

- **Data Warehouse:** Stores structured, curated data optimized for analytics, schema-on-write.
Modern lakehouse architecture combines both approaches.
-

Q27. Explain Slowly Changing Dimensions Type 2 step-by-step.

A27.

1. Detect changes in source.
 2. Mark existing record in dimension as expired (end_date).
 3. Insert new record with new surrogate key and start_date.
 4. Ensure queries use date range or current flag.
This preserves full historical changes.
-

Q28. What are the best practices for partitioning a Delta Table?

A28.

- Partition by high-cardinality, frequently queried columns (e.g., date).
 - Avoid over-partitioning (too many small files).
 - Use **OPTIMIZE + ZORDER** to improve performance.
 - Regularly compact small files.
This ensures efficient querying and storage management.
-

Q29. How do you troubleshoot pipeline failures?

A29.

- Use **ADF monitoring UI** to check activity runs.
 - Analyze logs in **Log Analytics**.
 - Check Spark UI in Databricks for transformations.
 - Implement alerts for proactive detection.
 - Retry with smaller batches if needed.
Systematic debugging ensures faster recovery.
-

Q30. Explain best practices for Spark cluster sizing.

A30.

- Size clusters based on dataset size, shuffle volume, and job complexity.
- Use **autoscaling clusters** for variable workloads.
- Avoid too many small executors (high overhead).
- Prefer fewer large executors for heavy jobs.
- Monitor via Spark UI and tune accordingly.
This optimizes cost vs performance.

EY – Data Engineer Interview Q&A

Q1. Explain data lakehouse architecture.

A1. A data lakehouse combines the scalability of data lakes with the reliability and performance of data warehouses. It stores raw, semi-structured, and structured data in one platform. Typically, it uses a **bronze-silver-gold (medallion) design**:

- Bronze: Raw data ingestion.
 - Silver: Cleaned, structured data.
 - Gold: Curated business-ready datasets.
- Technologies like **Delta Lake** provide ACID transactions, schema enforcement, and time travel, ensuring consistency and governance.

Q2. How do you design a scalable data ingestion pipeline?

A2. A scalable pipeline should:

- Use **event-driven ingestion** with Event Hubs/Kafka for streaming.
- Handle **batch ingestion** with ADF/Data Factory.
- Store in **ADLS Gen2** with partitioning.
- Apply **auto-scaling** in Databricks clusters.
- Leverage metadata-driven design for schema drift.
This ensures elasticity, reliability, and cost-efficiency.

Q3. What are key differences between Synapse Dedicated SQL Pool and Serverless Pool?

A3.

- **Dedicated Pool:** Pre-provisioned compute, best for predictable, high-performance workloads.
 - **Serverless Pool:** On-demand pay-per-query, best for ad-hoc queries. Dedicated ensures consistent performance, while serverless offers flexibility and cost savings for exploratory analytics.
-

Q4. How do you optimize PySpark jobs?

A4.

- Use **partitioning and bucketing** to reduce shuffle.
 - Avoid `collect()` and minimize wide transformations.
 - Use **broadcast joins** for small tables.
 - Cache/persist frequently reused DataFrames.
 - Enable **Adaptive Query Execution (AQE)**.
These practices reduce execution time and resource cost.
-

Q5. Explain broadcast join in Spark.

A5. A broadcast join replicates a small dataset across all nodes, avoiding shuffle.

Example: Joining a small customer dimension with a large sales fact.

```
from pyspark.sql.functions import broadcast
```

```
df_large.join(broadcast(df_small), "id")
```

This is highly efficient when one dataset fits into memory.

Q6. What is Adaptive Query Execution in Spark?

A6. AQE dynamically optimizes query plans at runtime based on actual data statistics.

It can:

- Change join strategies (e.g., shuffle → broadcast join).
 - Optimize skew by splitting large partitions.
 - Coalesce small partitions into fewer tasks.
- This results in more efficient query execution compared to static plans.
-

Q7. Explain CI/CD implementation for Databricks notebooks.

A7.

1. Store notebooks in GitHub/Azure Repos.
 2. Use **Databricks Repos** for syncing code.
 3. Configure **Azure DevOps pipelines** for validation and deployment.
 4. Use **Databricks CLI/REST API** for workspace imports/exports.
 5. Deploy to higher environments via release pipeline.
This ensures version control and repeatable deployments.
-

Q8. How do you handle changing source schema in ETL?

A8.

- Use **schema drift features** in ADF Data Flows.
 - In Delta Lake, enable mergeSchema.
 - Implement **metadata-driven ETL** to adapt dynamically.
 - Log and flag schema changes for validation.
This prevents pipeline breakage when columns are added/removed.
-

Q9. Explain mounting ADLS in Databricks.

A9. You can mount ADLS Gen2 using **dbutils.fs.mount()** with OAuth credentials or Managed Identity. Example:

```
dbutils.fs.mount(  
  source="abfss://container@account.dfs.core.windows.net/",  
  mount_point="/mnt/adls",  
  extra_configs={"fs.azure.account.auth.type": "OAuth"}  
)
```

This makes ADLS accessible like a local file system in Databricks.

Q10. How do you implement incremental load in Synapse Analytics?

A10. Incremental loads in Synapse can be achieved with:

- **Change Tracking** or **CDC** in source.

- **Watermarking** on datetime columns.
 - Using **MERGE statements** to update/insert records.
This ensures only changed rows are processed, reducing cost.
-

Q11. Explain trigger-based pipelines in ADF.

A11. Trigger-based pipelines start automatically. Types include:

- **Schedule Trigger** (e.g., daily refresh).
 - **Tumbling Window Trigger** (e.g., hourly batches).
 - **Event-based Trigger** (e.g., blob creation).
- They allow flexible automation of ETL processes.
-

Q12. How do you implement dynamic file loading in ADF?

A12. Use parameterized datasets and wildcards. Example:

- Parameterize file path (@pipeline().parameters.filename).
 - Use metadata-driven loops (ForEach activity).
 - Apply wildcard file matching (*.csv).
- This enables ingestion of multiple/dynamic files without hardcoding.
-

Q13. What is watermarking in streaming data?

A13. Watermarking tracks event-time progress and discards old data beyond a threshold. Example: withWatermark("eventTime", "10 minutes"). This handles late data arrivals while ensuring streaming jobs don't retain unbounded state.

Q14. Explain checkpointing in Spark Structured Streaming.

A14. Checkpointing saves progress and state to durable storage (e.g., ADLS). It ensures fault tolerance: if a job fails, it can restart from the last checkpoint instead of replaying from the beginning. This is essential for exactly-once processing.

Q15. How do you secure PII data in pipelines?

A15.

- Apply **data masking** or **tokenization**.
- Encrypt PII at rest and in transit.

- Restrict access with RBAC.
 - Use Key Vault for secret management.
 - Maintain audit logs for compliance.
This ensures GDPR/CCPA compliance.
-

Q16. Difference between Managed Identity and Service Principal.

A16.

- **Managed Identity:** Built-in identity for Azure resources. Simplifies authentication (no secret rotation needed).
 - **Service Principal:** An Azure AD application with client ID/secret used for authentication. Requires manual secret management.
Managed Identity is preferred for security and automation.
-

Q17. Explain audit logging in Azure.

A17. Azure provides logs for monitoring and compliance:

- **Activity Logs** for subscription-level events.
 - **Diagnostic Logs** for resource-level activity.
 - Integration with **Log Analytics** and **Azure Monitor**.
 - Export to **SIEM tools** for analysis.
These help track data access and system changes.
-

Q18. How do you monitor and alert on pipeline performance?

A18.

- Use **ADF Monitor** for activity runs.
 - Send logs to **Log Analytics**.
 - Create custom **metrics dashboards** in Azure Monitor.
 - Configure alerts via **Action Groups** (email, webhook, Logic Apps).
This ensures proactive monitoring of SLAs.
-

Q19. Explain Databricks cluster modes.

A19.

- **Standard Mode:** Jobs with full isolation.
 - **High Concurrency Mode:** Shared cluster with resource isolation.
 - **Single Node:** For development or lightweight jobs.
Choice depends on workload isolation, cost, and performance needs.
-

Q20. How do you optimize Delta Lake performance?

A20.

- Use **OPTIMIZE** to compact small files.
 - Apply **ZORDER** for faster queries.
 - Partition tables on frequently filtered columns.
 - Enable **caching** for hot datasets.
These best practices reduce latency for analytics queries.
-

Q21. What are ACID transactions in Delta Lake?

A21. Delta Lake supports **Atomicity, Consistency, Isolation, Durability**:

- **Atomicity:** All-or-nothing commits.
 - **Consistency:** Schema enforcement.
 - **Isolation:** Concurrent reads/writes handled safely.
 - **Durability:** Transaction logs ensure recovery.
This guarantees reliable data pipelines.
-

Q22. Explain schema validation in pipelines.

A22. Schema validation ensures source/target schemas match expectations. In ADF Data Flows, you can enforce schema checks. In Delta Lake, schema enforcement prevents mismatched data writes. This ensures data consistency and reduces errors.

Q23. What are different stages in ETL testing?

A23.

1. **Source-to-target validation.**
2. **Data completeness testing.**
3. **Transformation logic testing.**

4. **Performance testing.**

5. **Regression testing** after changes.

Testing ensures pipeline accuracy and reliability.

Q24. Explain your approach to pipeline version control.

A24. Use **Git integration** in ADF/Databricks. Store pipelines/notebooks as JSON or Python files. Implement branching strategy (dev/test/prod). Use CI/CD pipelines for deployment. This ensures collaborative development and rollback capability.

Q25. How do you orchestrate pipelines across multiple environments?

A25.

- Use **parameters/config files** for environment-specific values.
 - Deploy pipelines using CI/CD (ARM templates).
 - Separate data sources by environment (Dev/QA/Prod).
This ensures consistency across environments.
-

Q26. Explain data retention policies in Azure.

A26. Data retention policies define how long data is kept. In ADLS/Synapse, lifecycle policies can auto-delete/archive data after a set period. This reduces storage cost and ensures compliance with GDPR/industry rules.

Q27. What is Data Mesh architecture?

A27. Data Mesh is a decentralized approach where data ownership is distributed across domains. Each domain team owns its **data products**, exposing them via standard interfaces. It emphasizes **federated governance** and treats data as a product, improving scalability in large organizations.

Q28. Explain key considerations in designing a multi-region data lake.

A28.

- Data replication for high availability.
- Latency optimization by placing data near users.
- Compliance with data residency laws.

- Cross-region failover strategies.
 - Consistent governance policies across regions.
This ensures resilience and regulatory compliance.
-

Q29. How do you handle large fact tables in Synapse?

A29.

- Use **hash distribution** on frequently joined keys.
 - Apply **partitioning** on date or region columns.
 - Pre-aggregate data in summary tables.
 - Optimize with materialized views.
- These strategies improve query performance at scale.
-

Q30. Explain data deduplication strategies.

A30.

- Use **ROW_NUMBER() over partition** to keep latest record.
 - Apply dropDuplicates() in Spark.
 - Merge duplicates during ingestion with Delta Lake MERGE.
 - Create business rules (e.g., match on email + phone).
- Deduplication ensures clean, trustworthy datasets.
-

KPMG – Data Engineer Interview Q&A

Q1. How do you design an end-to-end data platform on Azure?

A1. An end-to-end Azure data platform typically includes:

- **Ingestion layer:** ADF/Data Factory, Event Hubs, IoT Hub.
- **Storage layer:** ADLS Gen2 as data lake.
- **Processing layer:** Databricks/Synapse for transformations.
- **Serving layer:** Synapse, Power BI for analytics.
- **Governance:** Purview, Key Vault, RBAC.

The design should follow medallion architecture for scalability and incorporate CI/CD and monitoring for production readiness.

Q2. Explain different data storage options in Azure.

A2.

- **Blob Storage:** General-purpose object storage.
 - **ADLS Gen2:** Optimized for analytics with hierarchical namespace.
 - **Cosmos DB:** NoSQL, globally distributed.
 - **Azure SQL Database:** Relational OLTP workloads.
 - **Synapse Analytics:** MPP data warehouse for OLAP.
Choice depends on workload type (structured vs unstructured, transactional vs analytical).
-

Q3. How do you create partition strategies in Spark?

A3. Partitioning improves parallelism. Strategies:

- **Range Partitioning** (e.g., by date).
 - **Hash Partitioning** (e.g., by customer ID).
 - **Custom Partitioning** via partitioners for special use cases.
Use `repartition()` for increasing partitions and `coalesce()` for reducing. The choice depends on data volume and query patterns.
-

Q4. Explain the difference between append-only and merge operations in Delta Lake.

A4.

- **Append-only:** New records are simply added (good for logs/events).
- **Merge:** Supports upserts (insert/update/delete) based on conditions. Example:

```
MERGE INTO target t
```

```
USING source s
```

```
ON t.id = s.id
```

```
WHEN MATCHED THEN UPDATE SET t.value = s.value
```

```
WHEN NOT MATCHED THEN INSERT *
```

Merge is essential for CDC and SCD handling.

Q5. How do you debug failed Spark jobs?

A5.

- Inspect **Spark UI** for stage/task breakdown.
 - Look for **skewed partitions** or shuffle issues.
 - Enable **event logs** and check executor logs.
 - Use **checkpointing** for streaming jobs.
- Systematic debugging helps isolate root cause quickly.

Q6. Explain performance tuning techniques in ADF.

A6.

- Increase **parallelism** in copy activity.
 - Use **staging (PolyBase/Bulk Copy)** for Synapse loads.
 - Optimize **integration runtime** configuration.
 - Enable **partitioned copy** for large tables.
 - Monitor using **ADF Monitor + Log Analytics**.
- These reduce latency and improve throughput.

Q7. What is data skew, and how do you handle it?

A7. Data skew occurs when some partitions have disproportionately more data than others. Solutions:

- **Salting:** Add random keys to distribute data.
 - **Repartitioning:** Balance partitions evenly.
 - **Broadcast joins:** Avoid skewed shuffle joins.
 - **Adaptive Query Execution** in Spark.
- These ensure workload balance across executors.

Q8. How do you configure pipeline retries in ADF?

A8. ADF allows setting **retry count** and **retry interval** at activity level. For time-windowed triggers, enable retry policy with backfill. Use **global error handling pipelines** for logging and reruns. This ensures resilience.

Q9. Explain triggers and event-based pipelines.

A9.

- **Schedule Trigger:** Fixed intervals.
- **Tumbling Window Trigger:** Recurring with retries.
- **Event-based Trigger:** Responds to blob creation or events from Event Grid.
Event-driven design supports near real-time ETL.

Q10. How do you optimize Synapse query performance?

A10.

- Use **distribution keys** wisely (hash vs round robin).
- Partition large fact tables.
- Create **materialized views** for aggregates.
- Apply **result set caching**.
- Use **CTAS** (Create Table As Select) for intermediate staging.
This ensures fast query response times.

Q11. What are the different notebook types in Databricks?

A11.

- **Python/Scala/SQL/R notebooks** for coding.
- **Dashboards** for visualization.
- **Collaborative notebooks** shared by teams.
Databricks also supports multi-language within the same notebook, improving flexibility.

Q12. Explain how to implement role-based access control in Azure.

A12. RBAC in Azure assigns roles to users/groups:

- **Reader/Contributor/Owner roles** at subscription/resource level.
- **Custom roles** for fine-grained access.

- Integration with **Azure AD** for authentication.
For data engineering, apply RBAC on ADLS folders, ADF pipelines, and Databricks clusters.
-

Q13. How do you handle multi-source joins in pipelines?

A13.

- In ADF: Use Data Flows with multiple sources.
 - In Spark: Load datasets, join using common keys, optimize with partitioning.
 - Handle schema mismatches with metadata mappings.
Ensuring consistent key management is crucial for accuracy.
-

Q14. What are different cluster modes in Spark?

A14.

- **Local Mode:** Runs on single machine.
 - **Standalone Mode:** Spark's own cluster manager.
 - **YARN Mode:** Runs on Hadoop cluster.
 - **Kubernetes Mode:** Uses K8s for containerized workloads.
 - **Databricks** abstracts cluster management.
Choice depends on deployment environment.
-

Q15. Explain PySpark Window functions.

A15. Window functions perform calculations across sets of rows. Example: Ranking, moving averages.

```
from pyspark.sql.window import Window
```

```
from pyspark.sql.functions import row_number
```

```
windowSpec = Window.partitionBy("dept").orderBy("salary")
```

```
df.withColumn("rank", row_number().over(windowSpec))
```

This assigns rank per department.

Q16. How do you test pipelines in lower environments?

A16.

- Use **sample data subsets**.
 - Parameterize connections for dev/test/prod.
 - Validate transformations with unit tests (e.g., PyTest for PySpark).
 - Automate deployment using CI/CD pipelines.
This ensures pipelines work before production rollout.
-

Q17. Explain Managed VNET in Data Factory.

A17. Managed VNET allows ADF to securely connect to resources without exposing them to the public internet. It provides:

- Private endpoint integration.
 - Data movement inside VNET.
 - Simplified security management.
This is essential for secure enterprise deployments.
-

Q18. What are Lake Databases in Synapse?

A18. Lake Databases are metadata layers on top of data in ADLS. They allow defining tables over raw/parquet data and querying with **serverless SQL pools**. This provides a schema-on-read experience without duplicating data.

Q19. Explain best practices for Delta Lake optimization.

A19.

- Compact small files with OPTIMIZE.
 - Use ZORDER for clustering.
 - Avoid over-partitioning.
 - Enable caching for hot tables.
 - Automate vacuuming of old files.
These keep Delta tables efficient and cost-friendly.
-

Q20. How do you debug a failing PolyBase query?

A20.

- Check external table definitions and file formats.
 - Validate file access permissions.
 - Look at error logs in Synapse/SQL DW.
 - Ensure compatibility of data types.
- Most failures are due to schema mismatch or file path issues.
-

Q21. Explain data cataloging in Unity Catalog.

A21. Unity Catalog provides centralized metadata management in Databricks. Features include:

- Organizing assets into catalogs, schemas, and tables.
 - Fine-grained permissions.
 - Column-level lineage and masking.
 - Auditing and governance integration.
- This ensures consistent discovery and governance of data assets.
-

Q22. How do you handle CDC (Change Data Capture) in ADF?

A22.

- Use source system CDC features (SQL Server CDC).
 - Track changes via timestamps/watermarks.
 - Use Delta Lake MERGE for upserts.
 - Maintain metadata tables for high-watermarks.
- This enables incremental data pipelines.
-

Q23. Explain your approach to designing ETL frameworks.

A23. A good ETL framework should be:

- **Metadata-driven** (dynamic source/target mappings).
- **Configurable** (parameterization).
- **Reusable** (modular components).

- **Fault-tolerant** (error handling, retries).
This reduces maintenance effort and increases scalability.
-

Q24. How do you secure Databricks clusters?

A24.

- Restrict network access with VNETs.
 - Enable cluster access control (only authorized users).
 - Use Key Vault-backed secrets for credentials.
 - Encrypt data in transit and at rest.
 - Enable audit logging.
This ensures secure compute environments.
-

Q25. Explain Audit and Governance implementation in Azure.

A25.

- Use **Azure Policy** for compliance.
 - Track activities via **Azure Monitor + Log Analytics**.
 - Catalog assets with **Purview**.
 - Enable **RBAC** and enforce naming standards.
 - Implement retention policies.
This ensures regulatory compliance and transparency.
-

Q26. How do you handle schema evolution in streaming pipelines?

A26.

- Use Delta Lake with mergeSchema.
 - Implement versioned schemas in metadata.
 - Apply backward-compatible changes (only adding columns).
 - Reject incompatible schema changes.
This prevents pipeline breakages in real-time workloads.
-

Q27. What are common bottlenecks in Delta Lake writes?

A27.

- Too many small files (write amplification).
- High shuffle during merges.
- Skewed partitions.
- Inefficient cluster sizing.

Best practices include compaction, ZORDER, and using efficient partition keys.

Q28. Explain checkpointing and recovery in Spark.

A28. Checkpointing stores intermediate state/data in durable storage. In case of failure, Spark recovers from checkpoint rather than recomputing. In streaming, it ensures exactly-once processing. Recovery reduces downtime and ensures reliability.

Q29. How do you automate ADF pipeline deployment?

A29.

- Store pipeline JSONs in Git repo.
 - Use Azure DevOps release pipeline.
 - Deploy via ARM templates or ADF REST API.
 - Apply environment-specific configs using parameters.
- This ensures consistency across Dev/QA/Prod.
-

Q30. Explain a real-world scenario where you optimized a data pipeline.

A30. Example: A retail pipeline loading daily sales data took 6 hours. Optimizations included:

- Switching from row-by-row inserts to bulk copy.
 - Using partitioned loads with PolyBase.
 - Replacing full reload with incremental CDC.
 - Optimizing joins in Databricks with broadcast join.
- Result: Pipeline runtime reduced from 6 hours to 45 minutes.